

```
#before running this please change the RUNTIME to GPU (Runtime -> Change runtime type -> set hardware accelerator to GPU)
#Mount our google drive
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Note : Use the drive link for the processed dataset

Start coding or [generate](#) with AI.

```
import glob
import torch
import torchvision
from torchvision import transforms
from torch.utils.data import DataLoader
from torch.utils.data.dataset import Dataset
import os
import numpy as np
import cv2
import matplotlib.pyplot as plt
#Check if the file is corrupted or not

def validate_video(vid_path,train_transforms):
    transform = train_transforms
    count = 20
    video_path = vid_path
    frames = []
    a = int(100/count)
    first_frame = np.random.randint(0,a)
    temp_video = video_path.split('/')[-1]
    for i,frame in enumerate(frame_extract(video_path)):
        frames.append(transform(frame))
        if(len(frames) == count):
            break
    frames = torch.stack(frames)
    frames = frames[:count]
    return frames

#extract a from from video
def frame_extract(path):
    vidObj = cv2.VideoCapture(path)
    success = 1
    while success:
        success, image = vidObj.read()
        if success:
            yield image

im_size = 112
mean = [0.485, 0.456, 0.406]
std = [0.229, 0.224, 0.225]

train_transforms = transforms.Compose([
    transforms.ToPILImage(),
    transforms.Resize((im_size,im_size)),
    transforms.ToTensor(),
    transforms.Normalize(mean,std)])

video_fil = glob.glob('/content/drive/My Drive/Mtech_Datasets/Celeb_fake_face/*.mp4')
video_fil += glob.glob('/content/drive/My Drive/Mtech_Datasets/Celeb_real_face/*.mp4')
video_fil += glob.glob('/content/drive/My Drive/Mtech_Datasets/DFDC_FAKE_Face/*.mp4')
video_fil += glob.glob('/content/drive/My Drive/Mtech_Datasets/DFDC_REAL_Face/*.mp4')
video_fil += glob.glob('/content/drive/My Drive/Mtech_Datasets/FF_Face/*.mp4')
print("Total no of videos :", len(video_fil))
print(video_fil)
count = 0;
for i in video_fil:
    try:
        count+=1
        validate_video(i,train_transforms)
    except:
        print("Number of video processed: " , count , " Remaining : " , (len(video_fil) - count))
        print("Corrupted video is : " , i)
        continue
print((len(video_fil) - count))
```

Total no of videos : 5660

['/content/drive/My Drive/Mtech_Datasets/Celeb_fake_face/id34_id32_0002.mp4', '/content/drive/My Drive/Mtech_Data
Number of video processed: 70 Remaining : 5590

Corrupted video is : /content/drive/My Drive/Mtech_Datasets/Celeb_fake_face/id53_id50_0007.mp4
 Number of video processed: 760 Remaining : 4900
 Corrupted video is : /content/drive/My Drive/Mtech_Datasets/Celeb_real_face/id37_0009.mp4

```
import json
import glob
import numpy as np
import cv2
import copy
import random
video_files = glob.glob('/content/drive/My Drive/Mtech_Datasets/Celeb_fake_face/*.mp4')
video_files += glob.glob('/content/drive/My Drive/Mtech_Datasets/Celeb_real_face/*.mp4')
video_files += glob.glob('/content/drive/My Drive/Mtech_Datasets/DFDC_FAKE_Face/*.mp4')
video_files += glob.glob('/content/drive/My Drive/Mtech_Datasets/DFDC_REAL_Face/*.mp4')
video_files += glob.glob('/content/drive/My Drive/Mtech_Datasets/FF_Face/*.mp4')
random.shuffle(video_files)
random.shuffle(video_files)
frame_count = []

for video_file in video_files:
    cap = cv2.VideoCapture(video_file)
    if(int(cap.get(cv2.CAP_PROP_FRAME_COUNT))<100):
        video_files.remove(video_file)
        continue
    frame_count.append(int(cap.get(cv2.CAP_PROP_FRAME_COUNT)))
print("frames are " , frame_count)
print("Total no of video: " , len(frame_count))
print('Average frame per video:',np.mean(frame_count))
```

frames are [148, 148, 148, 148, 148, 148, 148, 148, 148, 148, 148, 148, 148, 148, 147, 148, 148, 148,
 Total no of video: 5616
 Average frame per video: 147.76834045584044

```
# load the video name and labels from csv
import torch
import torchvision
from torchvision import transforms
from torch.utils.data import DataLoader
from torch.utils.data.dataset import Dataset
import os
import numpy as np
import cv2
import matplotlib.pyplot as plt
class video_dataset(Dataset):
    def __init__(self,video_names,labels,sequence_length = 60,transform = None):
        self.video_names = video_names
        self.labels = labels
        self.transform = transform
        self.count = sequence_length
    def __len__(self):
        return len(self.video_names)
    def __getitem__(self,idx):
        video_path = self.video_names[idx]
        frames = []
        a = int(100/self.count)
        first_frame = np.random.randint(0,a)
        temp_video = video_path.split('/')[1]
        #print(temp_video)
        label = self.labels.iloc[(labels.loc[labels["file"] == temp_video].index.values[0]),1]
        if(label == 'FAKE'):
            label = 0
        if(label == 'REAL'):
            label = 1
        for i,frame in enumerate(self.frame_extract(video_path)):
            frames.append(self.transform(frame))
            if(len(frames) == self.count):
                break
        frames = torch.stack(frames)
        frames = frames[:self.count]
        #print("length:" , len(frames), "label",label)
        return frames,label
    def frame_extract(self,path):
        vidObj = cv2.VideoCapture(path)
        success = 1
        while success:
            success, image = vidObj.read()
            if success:
                yield image
#plot the image
def im_plot(tensor):
```

```

image = tensor.cpu().numpy().transpose(1,2,0)
b,g,r = cv2.split(image)
image = cv2.merge((r,g,b))
image = image*[0.22803, 0.22145, 0.216989] + [0.43216, 0.394666, 0.37645]
image = image*255.0
plt.imshow(image.astype(int))
plt.show()

```

```

#count the number of fake and real videos
def number_of_real_and_fake_videos(data_list):
    header_list = ["file","label"]
    lab = pd.read_csv('/content/drive/My Drive/Model Creation/labels/Gobal_metadata.csv',names=header_list)
    fake = 0
    real = 0
    for i in data_list:
        temp_video = i.split('/')[1]
        label = lab.iloc[(lab.loc[lab["file"] == temp_video].index.values[0]),1]
        if(label == 'FAKE'):
            fake+=1
        if(label == 'REAL'):
            real+=1
    return real,fake

```

```

# load the labels and video in data loader
import random
import pandas as pd
from sklearn.model_selection import train_test_split

header_list = ["file","label"]
labels = pd.read_csv('/content/drive/My Drive/Model Creation/labels/Gobal_metadata.csv',names=header_list)
print(labels)
train_videos = video_files[:int(0.8*len(video_files))]
valid_videos = video_files[int(0.8*len(video_files)):]
print("train : " , len(train_videos))
print("test : " , len(valid_videos))
#train_videos,valid_videos = train_test_split(data,test_size = 0.2)
print(train_videos)

print("TRAIN: ", "Real:",number_of_real_and_fake_videos(train_videos)[0]," Fake:",number_of_real_and_fake_videos(
print("TEST: ", "Real:",number_of_real_and_fake_videos(valid_videos)[0]," Fake:",number_of_real_and_fake_videos(

im_size = 112
mean = [0.485, 0.456, 0.406]
std = [0.229, 0.224, 0.225]

train_transforms = transforms.Compose([
    transforms.ToPILImage(),
    transforms.Resize((im_size,im_size)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees=15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue
    transforms.ToTensor(),
    transforms.Normalize(mean,std)])

test_transforms = transforms.Compose([
    transforms.ToPILImage(),
    transforms.Resize((im_size,im_size)),
    transforms.ToTensor(),
    transforms.Normalize(mean,std)])

train_data = video_dataset(train_videos,labels,sequence_length = 10,transform = train_transforms)
#print(train_data)
val_data = video_dataset(valid_videos,labels,sequence_length = 10,transform = train_transforms)
train_loader = DataLoader(train_data,batch_size = 4,shuffle = True,num_workers = 4)
valid_loader = DataLoader(val_data,batch_size = 4,shuffle = True,num_workers = 4)
image,label = train_data[0]
im_plot(image[0,:,:,:])

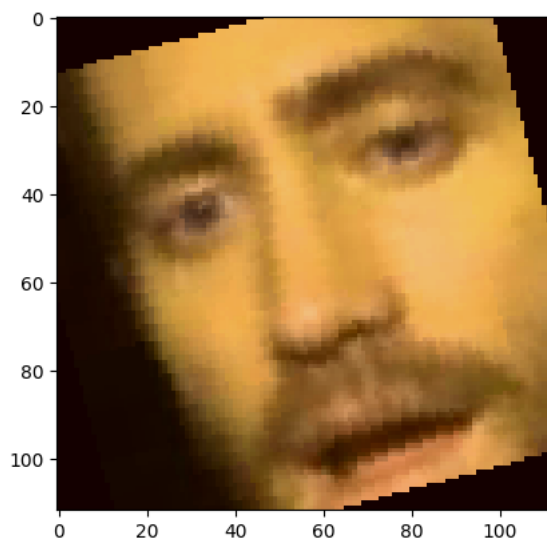
```

```

      file label
0      000.mp4 REAL
1      000_003.mp4 FAKE
2      001.mp4 REAL
3      001_870.mp4 FAKE
4      002.mp4 REAL
...      ...
23181 ybbrkacebd.mp4 REAL
23182 ybnucgidtu.mp4 REAL
23183 yexeazbqig.mp4 REAL
23184 yhylappzid.mp4 REAL
23185 yljecirelf.mp4 REAL

[23186 rows x 2 columns]
train : 4510
test : 1128
['/content/drive/My Drive/Mtech_Datasets/Celeb_real_face/id2_0002.mp4', '/content/drive/My Drive/Mtech_Datasets/D
TRAIN: Real: 2318 Fake: 2192
/usr/local/lib/python3.11/dist-packages/torch/utils/data/dataloader.py:624: UserWarning: This DataLoader will cre
warnings.warn(
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0
TEST: Real: 591 Fake: 537

```



```

import torch
import torchvision
from torchvision import transforms
from torch.utils.data import DataLoader
from torch.utils.data.dataset import Dataset
import os
import numpy as np
import cv2
import matplotlib.pyplot as plt
# Import necessary modules
import torch.nn as nn # Import nn for neural network components
from torchvision import models # Import models for pre-trained models
import torch.optim as optim # Import optim for optimizers
class DeepfakeDetector(nn.Module):
    def __init__(self, num_classes=2):
        super().__init__()

        # CNN Feature Extractor (ResNeXt50)
        base_model = models.resnet50(pretrained=True)
        self.cnn = nn.Sequential(*list(base_model.children())[:-2])
        self.avgpool = nn.AdaptiveAvgPool2d(1)

        # Temporal Modeling (BiGRU)
        self.gru = nn.GRU(
            input_size=2048,
            hidden_size=1024,
            num_layers=2, # Try reducing to 1 layer
            bidirectional=True,
            batch_first=True,
            dropout=0.2 # Slightly reduced dropout
        )

        # Attention Mechanism
        self.attention = nn.MultiheadAttention(
            embed_dim=2048, # 1024*2 (bidirectional)
            num_heads=8,

```

```

        dropout=0.4,      # Slightly reduced dropout
        batch_first=True
    )

    # Classification Head
    self.classifier = nn.Sequential(
        nn.LayerNorm(2048),
        nn.Dropout(0.4),    # Adjusted dropout
        nn.Linear(2048, 512),
        nn.GELU(),
        nn.Dropout(0.4),    # Adjusted dropout
        nn.Linear(512, num_classes)
    )

    def forward(self, x):
        batch_size, seq_len, c, h, w = x.shape

        # CNN Features
        x = x.view(batch_size*seq_len, c, h, w)
        x = self.cnn(x)
        x = self.avgpool(x)
        x = x.view(batch_size, seq_len, -1)

        # BiGRU
        gru_out, _ = self.gru(x)

        # Attention
        attn_out, _ = self.attention(gru_out, gru_out, gru_out)
        context = gru_out + attn_out # Residual connection

        # Classification
        context = torch.mean(context, dim=1)
        return self.classifier(context)

```

```

model = DeepfakeDetector()
criterion = nn.CrossEntropyLoss(label_smoothing=0.1).cuda()
LR = 1e-4
optimizer = optim.AdamW(model.parameters(), lr=LR, weight_decay=1e-4)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, 'max', patience=3, factor=0.5)

```

```

/usr/local/lib/python3.11/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained'
warnings.warn(
/usr/local/lib/python3.11/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a wei
warnings.warn(msg)

```

```

import torch
from torch.autograd import Variable
import time
import os
import sys
import os

class AverageMeter(object):
    """Computes and stores the average and current value"""
    def __init__(self):
        self.reset()
    def reset(self):
        self.val = 0
        self.avg = 0
        self.sum = 0
        self.count = 0

    def update(self, val, n=1):
        self.val = val
        self.sum += val * n
        self.count += n
        self.avg = self.sum / self.count

def calculate_accuracy(outputs, targets):
    batch_size = targets.size(0)
    _, pred = outputs.topk(1, 1, True)
    pred = pred.t()
    correct = pred.eq(targets.view(1, -1))
    n_correct_elems = correct.float().sum().item()
    return 100 * n_correct_elems / batch_size

def train_epoch(epoch, num_epochs, data_loader, model, criterion, optimizer):
    # Move the model to the GPU
    if torch.cuda.is_available():

```

```

        model.cuda()

    model.train()
    losses = AverageMeter()
    accuracies = AverageMeter()
    t = []
    for i, (inputs, targets) in enumerate(data_loader):
        if torch.cuda.is_available():
            targets = targets.type(torch.cuda.LongTensor)
            inputs = inputs.cuda()
            outputs = model(inputs)

            loss = criterion(outputs, targets.type(torch.cuda.LongTensor))
            acc = calculate_accuracy(outputs, targets.type(torch.cuda.LongTensor))
            losses.update(loss.item(), inputs.size(0))
            accuracies.update(acc, inputs.size(0))
            optimizer.zero_grad()
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0) # Gradient Clipping
            optimizer.step()
            sys.stdout.write(
                "\r[Epoch %d/%d] [Batch %d / %d] [Loss: %f, Acc: %.2f%%]"
                % (
                    epoch,
                    num_epochs,
                    i,
                    len(data_loader),
                    losses.avg,
                    accuracies.avg,
                )
            )
    torch.save(model.state_dict(), '/content/drive/My Drive/Mtech_Datasets/new_combined_checkpoint.pt')
    return losses.avg, accuracies.avg

def test(epoch, model, data_loader, criterion):

    model.eval()
    losses = AverageMeter()
    accuracies = AverageMeter()
    pred = []
    true = []
    count = 0
    with torch.no_grad():
        for i, (inputs, targets) in enumerate(data_loader):
            if torch.cuda.is_available():
                targets = targets.cuda().type(torch.cuda.FloatTensor)
                inputs = inputs.cuda()
                outputs = model(inputs)
                loss = torch.mean(criterion(outputs, targets.type(torch.cuda.LongTensor)))
                acc = calculate_accuracy(outputs, targets.type(torch.cuda.LongTensor))
                _, p = torch.max(outputs, 1)
                true += (targets.type(torch.cuda.LongTensor)).detach().cpu().numpy().reshape(len(targets)).tolist()
                pred += p.detach().cpu().numpy().reshape(len(p)).tolist()
                losses.update(loss.item(), inputs.size(0))
                accuracies.update(acc, inputs.size(0))
            ...
            sys.stdout.write(
                "\r[Batch %d / %d] [Loss: %f, Acc: %.2f%%]"
                % (
                    i,
                    len(data_loader),
                    losses.avg,
                    accuracies.avg,
                )
            )
            ...
    print('\nAccuracy {}'.format(accuracies.avg))
    return true, pred, losses.avg, accuracies.avg

```

```

from sklearn.metrics import confusion_matrix
#learning rate
lr = 1e-5 #0.001
#number of epochs
num_epochs = 10

optimizer = torch.optim.Adam(model.parameters(), lr= lr, weight_decay = 1e-5)

criterion = nn.CrossEntropyLoss().cuda()
train_loss_avg = []
train_accuracy = []
test_loss_avg = []
test_accuracy = []

```

```

for epoch in range(1,num_epochs+1):
    l , acc = train_epoch(epoch,num_epochs,train_loader,model,criterion,optimizer)
    train_loss_avg.append(l)
    train_accuracy.append(acc)
    true,pred,tl,t_acc = test(epoch,model,valid_loader,criterion)
    test_loss_avg.append(tl)
    test_accuracy.append(t_acc)

```

```

/usr/local/lib/python3.11/dist-packages/torch/utils/data/dataloader.py:624: UserWarning: This DataLoader will create
warnings.warn(
[Epoch 1/10] [Batch 1127 / 1128] [Loss: 0.678314, Acc: 60.89%]
Accuracy 70.92198581560284
[Epoch 2/10] [Batch 1127 / 1128] [Loss: 0.589259, Acc: 72.13%]
Accuracy 73.49290780141844
[Epoch 3/10] [Batch 1127 / 1128] [Loss: 0.551713, Acc: 76.52%]
Accuracy 79.34397163120568
[Epoch 4/10] [Batch 1127 / 1128] [Loss: 0.588059, Acc: 79.73%]
Accuracy 80.67375886524823
[Epoch 5/10] [Batch 1127 / 1128] [Loss: 0.576503, Acc: 80.16%]
Accuracy 80.93971631205673
[Epoch 6/10] [Batch 1127 / 1128] [Loss: 0.604400, Acc: 81.55%]
Accuracy 80.05319148936171
[Epoch 7/10] [Batch 1127 / 1128] [Loss: 0.582639, Acc: 82.64%]
Accuracy 82.5354609929078
[Epoch 8/10] [Batch 166 / 1128] [Loss: 0.564931, Acc: 82.19%]

```

```

print(test_accuracy)
print(true,pred)
print(train_loss_avg,test_loss_avg)
print(train_accuracy,test_accuracy)

```

```

[83.51063829787235, 82.44680851063829, 82.26950354609929, 82.18085106382979, 83.68794326241135, 85.37234042553192
[1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 1, 1, 0, 1, 0
[0.5768300305617283, 0.5355293473605162, 0.5727720267187904, 0.5098735815534645, 0.4919037941357734, 0.4719384162
[85.1219512195122, 85.8980044345898, 86.0310421286031, 87.73835920177383, 88.51441241685144, 89.06873614190687, 9

```

```

import numpy as np
import pandas as pd
import seaborn as sn
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix,precision_score, recall_score, f1_score

```

```

def print_confusion_matrix(y_true, y_pred):
    cm = confusion_matrix(y_true, y_pred)

```

```

    print('True Negative (Fake predicted as Fake): ', cm[0][0])
    print('False Positive (Fake predicted as Real): ', cm[0][1])
    print('False Negative (Real predicted as Fake): ', cm[1][0])
    print('True Positive (Real predicted as Real): ', cm[1][1])
    print('\n')

```

```

    labels = ['Fake', 'Real']
    df_cm = pd.DataFrame(cm, index=labels, columns=labels)

```

```

    sn.set(font_scale=1.2)
    sn.heatmap(df_cm, annot=True, fmt='d', cmap="magma", annot_kws={"size": 14})
    plt.ylabel('Actual label', fontsize=16)
    plt.xlabel('Predicted label', fontsize=16)
    plt.xticks(fontsize=12)
    plt.yticks(fontsize=12, rotation=0)
    plt.tight_layout()
    plt.show()

```

```

    acc = (cm[0][0] + cm[1][1]) / np.sum(cm)
    print(f"Calculated Accuracy: {acc * 100:.2f}%")

```

```

print(confusion_matrix(true,pred))
print_confusion_matrix(true,pred)
precision = precision_score(true, pred)
recall = recall_score(true, pred)
f1 = f1_score(true, pred)

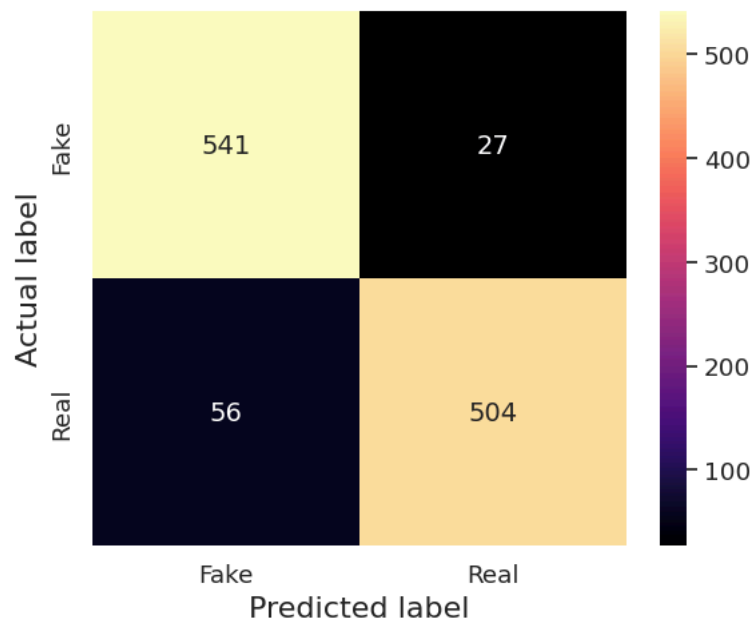
```

```

print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")
print(f"F1 Score: {f1:.2f}")

```

True Negative (Fake -> Fake): 541
False Positive (Fake -> Real): 27
False Negative (Real -> Fake): 56
True Positive (Real -> Real): 504



Calculated Accuracy: 92.64%
Precision: 0.95
Recall: 0.9000
F1 Score: 0.9239

```
from sklearn.metrics import roc_auc_score
roc_auc = roc_auc_score(true, pred)
print(f"ROC AUC Score: {roc_auc:.4f}")
```

ROC AUC Score: 0.925

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.